

10 从动力学角度看优化算法（五）：为什么学习率不宜过小？

Oct By 苏剑林 | 2020-10-10 | 70587位读者 | [Kimi 引用](#)

本文的主题是“为什么我们需要有限的学习率”，所谓“有限”，指的是不大也不小，适中即可，太大容易导致算法发散，这不难理解，但为什么太小也不好呢？一个容易理解的答案是，学习率过小需要迭代的步数过多，这是一种没有必要的浪费，因此从“节能”和“加速”的角度来看，我们不用过小的学习率。但如果不考虑算力和时间，那么过小的学习率是否可取呢？Google最近发表在Arxiv上的论文《[Implicit Gradient Regularization](#)》试图回答了这个问题，它指出**有限的学习率隐式地给优化过程带来了梯度惩罚项**，而这个梯度惩罚项对于提高泛化性能是有帮助的，因此哪怕不考虑算力和时间等因素，也不应该用过小的学习率。

对于梯度惩罚，本博客已有过多次讨论，在文章《[对抗训练浅谈：意义、方法和思考（附Keras实现）](#)》和《[泛化性乱弹：从随机噪声、梯度惩罚到虚拟对抗训练](#)》中，我们就分析了对抗训练一定程度上等价于对输入的梯度惩罚，而文章《[我们真的需要把训练集的损失降低到零吗？](#)》介绍的Flooding技巧则相当于对参数的梯度惩罚。总的来说，不管是对输入还是对参数的梯度惩罚，都对提高泛化能力有一定帮助。

降得最快的方向

该论文跟[这个系列的文章](#)一样，将优化过程看成是求解微分方程。回顾之前的博文《[从动力学角度看优化算法（三）：一个更整体的视角](#)》，设损失函数为 $L(\theta)$ ，我们将 θ 看成是看成是沿着某种时间参数 t 变化的轨迹 $\theta(t)$ ，现在我们考虑它的变化率：

$$\frac{d}{dt}L(\theta(t)) = \left\langle \nabla_{\theta}L(\theta(t)), \dot{\theta}(t) \right\rangle$$

我们希望 $L(\theta(t))$ 随着时间的变化是递减的（Loss越小越好），所以希望上式小于0，当模长 $|\dot{\theta}(t)|$ 固定时，上式右端的最小值在梯度的反方向 $-\nabla_{\theta}L(\theta(t))$ 取到，所以我们说梯度的负方向下降得最快方向。简单期间，我们可以直接令

$$\dot{\theta}(t) = -\nabla_{\theta}L(\theta(t)) \triangleq -g(\theta(t))$$

那么求解参数 θ 就转化为求解上述常微分方程组，这也是“从动力学角度看优化算法”这个系列的基本出发点。

藏在学习率中的正则

然而，实际的问题是，我们没法真正去求解微分方程组(2)，我们只能用数值迭代，比如采用最简单的欧拉法，得到

$$\theta_{t+\gamma} = \theta_t - \gamma g(\theta_t)$$

这其实就是最朴素的梯度下降法，其中 γ 也就是我们常说的学习率。上式本质上就是一个差分方程。

可以想象，从 $t=0$ 出发，得到的点 $\theta_{\gamma}, \theta_{2\gamma}, \theta_{3\gamma}, \dots$ 与方程组(2)的精确解 $\theta(\gamma), \theta(2\gamma), \theta(3\gamma), \dots$ 会有一定的出入。如何衡量出入到什么程度呢？不妨这样想象， $\theta_{\gamma}, \theta_{2\gamma}, \theta_{3\gamma}, \dots$ 其实也是某个类似(2)的微分方程组的精确解，只不过对应的 $g(\theta(t))$ 换成了某个新的 $\tilde{g}(\theta_t)$ ，我们比较 $\tilde{g}(\theta_t)$ 与 $g(\theta(t))$ 的差异就好了。

经推导，如果仅保留到 γ 的一阶项，那么有

$$\tilde{g}(\theta_t) = g(\theta_t) + \frac{\gamma}{4} \nabla_{\theta} \|g(\theta_t)\|^2 = \nabla_{\theta} \left(L(\theta_t) + \frac{1}{4} \gamma \|\nabla_{\theta} L(\theta_t)\|^2 \right)$$

推导过程我们放在下一节。可以看到，其实就相当于往损失函数里边加入了梯度惩罚形式的正则项 $\frac{1}{4} \gamma \|\nabla_{\theta} L(\theta)\|^2$ ，而梯度惩罚项有助于模型到达更加平缓的区域，有利于提高泛化性能。这也就是说，离散化的迭代过程隐式地带来了梯度惩罚项，反而是对模型的泛化是有帮助的，而如果 $\gamma \rightarrow 0$ ，这个隐式的惩罚则会变弱甚至消失。

因此，结论就是**学习率不宜过小，较大的学习率不仅有加速收敛的好处，还有提高模型泛化能力的好处**。当然，可能有些读者会想，我直接把梯度惩罚加入到loss中，是不是就可以用足够小的学习率了？理论上确实是的，原论文将梯度惩罚加入到loss中的做法，称为“显式梯度惩罚”。

差分方程到微分方程

对于差分方程到微分方程的转换，我们可以用普通的“摄动法”来求解，本博客也有过简单介绍（可以查看标签“[摄动](#)”）。不过更漂亮的解法是直接利用算符的级数运算来做，参考之前的文章《[算符的艺术：差分、微分与伯努利数](#)》。

我们用泰勒级数展开 $\theta_{t+\gamma}$ ：

$$\theta_{t+\gamma} = \theta_t + \gamma \dot{\theta}_t + \frac{1}{2} \gamma^2 \ddot{\theta}_t + \frac{1}{6} \gamma^3 \theta_t^{(3)} + \dots$$

如果将对 t 求导的运算记为 D ，那么上式实际上是

$$\theta_{t+\gamma} = \left(1 + \gamma D + \frac{1}{2}\gamma^2 D^2 + \frac{1}{6}\gamma^3 D^3 + \dots\right) \theta_t = e^{\gamma D} \theta_t$$

所以差分方程(3)可以写为

$$\left(e^{\gamma D} - 1\right) \theta_t = -\gamma g(\theta_t)$$

跟常规的代数运算一样，我们有

$$\begin{aligned} D\theta_t &= -\gamma \left(\frac{D}{e^{\gamma D} - 1} \right) g(\theta_t) \\ &= -\left(1 - \frac{1}{2}\gamma D + \frac{1}{12}\gamma^2 D^2 - \frac{1}{720}\gamma^4 D^4 + \dots\right) g(\theta_t) \end{aligned}$$

等号左端就是 $\dot{\theta}_t$ ，因此等号右端就是 $-\tilde{g}(\theta_t)$ 的表达式了，保留到一阶项为

$$-\left(1 - \frac{1}{2}\gamma D\right) g(\theta_t) = -g(\theta_t) + \frac{1}{2}\gamma \frac{d}{dt} g(\theta_t) = -g(\theta_t) + \frac{1}{2}\gamma \nabla_{\theta} g(\theta_t) \dot{\theta}_t$$

也就是

$$\begin{aligned} \dot{\theta}_t &= -g(\theta_t) + \frac{1}{2}\gamma \nabla_{\theta} g(\theta_t) \dot{\theta}_t \\ &= -g(\theta_t) + \frac{1}{2}\gamma \nabla_{\theta} g(\theta_t) \left[-g(\theta_t) + \frac{1}{2}\gamma \nabla_{\theta} g(\theta_t) \dot{\theta}_t \right] \\ &= -g(\theta_t) - \frac{1}{2}\gamma \nabla_{\theta} g(\theta_t) g(\theta_t) \quad (\text{略去二阶项}) \\ &= -g(\theta_t) - \frac{1}{4}\gamma \nabla_{\theta} \|g(\theta_t)\|^2 \end{aligned}$$

所以一阶的 $\tilde{g}(\theta_t) = g(\theta_t) + \frac{1}{4}\gamma \nabla_{\theta} \|g(\theta_t)\|^2$ ，推导完毕。

例行公事的小总结

深度学习的发展和普及离不开基于梯度下降的优化器的成功应用，而梯度下降为何能如此成功，依然还没得到深刻的解释。众多研究人员在“炼丹”过程中，多多少少也能总结出一些不知道为什么有效的“奇技淫巧”出来，诸如batch_size该取多大、学习率该怎么调，估计每个人也有自己的经验。

对于“学习率不能过小”这个现象，大家应该都有所体会，很多时候可能已经默认作为一个“常识”使用，而懒得思考背后的原理了。Google的这篇论文则为理解这个现象提供了一个可能的解释：适当而不是过小的学习率能为优化过程带来隐式的梯度惩罚项，有助于收敛到更平稳的区域。笔者认为其分析过程还是值得参考学习的。

转载到请包括本文地址： <https://kexue.fm/archives/7787>

更详细的转载事宜请参考：《科学空间FAQ》

如果您需要引用本文，请参考：

苏剑林. (Oct. 10, 2020). 《从动力学角度看优化算法（五）：为什么学习率不宜过小？》 [Blog post]. Retrieved from <https://kexue.fm/archives/7787>

```
@online{kexuefm-7787,
  title={从动力学角度看优化算法（五）：为什么学习率不宜过小? },
  author={苏剑林},
  year={2020},
  month={Oct},
  url={\url{https://kexue.fm/archives/7787}},
}
```